

A certifying algorithm for the path cover problem on interval graphs*

Maw-Shang Chang

Department of Computer Science and Information Engineering

National Chung Cheng University

Ming-Hsiung, Chiayi 621, Taiwan, R.O.C.

E-mail: mschang@cs.ccu.edu.tw

February 29, 2008

Abstract

A certifying algorithm for a problem is an algorithm that provides a certificate with each answer that it produces. The certificate is a piece of evidence that proves that the answer has not been compromised by a bug in the implementation. A *path cover* of a graph $G = (V, E)$ is a family of vertex-disjoint paths that covers all vertices in V . Given a graph G , the *path cover problem* is to find a path cover of minimum cardinality. This short note presents an $O(n)$ -time certifying algorithm for the path cover problem on interval graphs given a set of n intervals with endpoints sorted.

Keywords: Path cover; Interval graphs; Circular-arc graphs; Certifying Algorithms

1 Introduction

A *certifying algorithm* for a problem is an algorithm that provides a certificate with each answer that it produces. An *authentication algorithm* is an algorithm that confirms the validity of the answer by checking the certificate. The name “certifying algorithm” was coined in [16]. For example, an implementation of a certifying algorithm

testing whether an input graph is bipartite outputs an odd cycle of the input graph whenever it claims the input graph is not bipartite and outputs two disjoint independent sets whose union is the vertex set of the input graph if it claims that the input graph is bipartite indeed. An authentication program can verify the correctness of an answer claiming the input graph is not bipartite by checking whether the certificate is an odd cycle of the input graph indeed; and verify the correctness of an answer claiming the input graph is bipartite by checking whether the certificate does consist of two disjoint independent sets whose union is the vertex set of the input graph. Usually we prove the correctness of our algorithms after we design it. The purpose of a certificate produced by a certifying algorithm is not to prove the correctness of the algorithm. It’s for implementation. When we implement a valid algorithm, we are not sure that the implementation is error free. One way to see the correctness of the implementation is to output a certificate for each answer it produces and let an authentication program verify the correctness

*The research was supported in part by the National Science Council of Taiwan under grant no. NSC96-2221-E-194-045-MY3.

of the answer by checking the certificate. The certificate has practical value when the implementation of an algorithm for solving the problem from scratch is difficult but the implementation of the authentication algorithm is easier. For more aspects on certifying algorithms we refer the readers to [17, 20].

A *recognition algorithm* is an algorithm that decides whether some given input (graph, geometrical object, picture, etc.) has a certain property. Such an algorithm accepts the input if it has the property or rejects it if it does not. There are some certifying recognition algorithms appeared in the literatures recently [10, 17, 14]. In fact some recognition algorithms published before the term of certifying algorithms appeared are certifying algorithm already. We give a certifying algorithms for an optimization problem, i.e., the path cover problem on interval graphs.

All graphs considered in this paper are finite and undirected, without loops or multiple edges. Throughout this paper, let n and m denote the numbers of vertices and edges of graph G , respectively. A *Hamiltonian cycle* in a graph is a (simple) cycle in which each vertex of the graph appears exactly once. A *Hamiltonian path* in a graph is a (simple) path with the same property. The *Hamiltonian cycle problem* involves testing whether or not a graph contains a Hamiltonian cycle. The *Hamiltonian path problem* involves testing whether or not a graph contains a Hamiltonian path. These two problems are NP-complete for general graphs [8]. The same holds true for bipartite graphs [15], split graphs [9], circle graphs [6], undirected path graphs [2] and grid graphs [13].

A *path cover* of a graph $G = (V, E)$ is a family of vertex-disjoint paths that covers all vertices in V . Given a graph G , the *path cover problem* is to find a path cover of minimum number, denoted by $\pi(G)$, of paths. This problem is NP-hard for arbitrary graphs [8] since it contains the Hamiltonian path problem as a special case. The path cover problem has found some applications in database designing, networking, establishing ring protocol, VLSI designing, code optimization [3], and mapping parallel programs into parallel architectures [21, 22]. Polynomial-time algorithms were known only for a few special classes of graphs including trees [21], block graphs [5, 24, 25], interval graphs [1, 4], circular-arc graphs [11], cographs [18], bipartite distance-hereditary graphs [26], bipartite permutation graphs [23], cocomparability graphs [7] and distance-hereditary graphs [12].

Arikati and Rangan presented an $O(n+m)$ time algorithm to solve the path cover problem on interval graphs [1]. Chang et al. proposed $O(n)$ -time algorithms for both the Hamiltonian cycle and path cover problems on interval graphs given a set of n sorted intervals [4].

2 A certifying algorithm for the path cover problem on interval graphs

We start with some notation. Let $G = (V, E)$ be a graph without isolated vertices. A path P of length $|P|$ in G , denoted by $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_{|P|}$, is a sequence $(v_1, v_2, \dots, v_{|P|})$ of vertices such $(v_i, v_{i+1}) \in E$ for $1 \leq i < |P|$. The first and last vertices visited by P are called the *path-start* and *path-end* of P , denoted by $start(P)$ and $end(P)$,

respectively. We use $v_i \in P$ to denote “ P visits v_i ”. In addition, we will use P to refer to the set of vertices visited by path P if it is understood without ambiguity.

For two sets A and B we write $A - B$ instead of $A \setminus B$. We write $A \subseteq B$ if A is a subset of B with possible equality and we write $A \subset B$ if A is a subset of B and $A \neq B$. For a set A and an element x we write $A - x$ instead of $A \setminus \{x\}$ and $A + x$ instead of $A \cup \{x\}$.

For a graph $G = (V, E)$ and a subset $S \subseteq V$ of vertices, we write $G[S]$ for the subgraph of G induced by S , $G - S$ for the subgraph $G[V - S]$, i.e., the subgraph induced by $V - S$. For a vertex v we write $G - v$ rather than $G - \{v\}$. Let S and C be two disjoint subsets of vertices in G such that S is nonempty. We say that S is an *island with respect to C in G* or an *island in $G - C$* if no vertex in S is adjacent to any vertex of $V - (C \cup S)$ in G . By the above definition, an island S with respect to C in G is not empty and it contains at least one connected component in $G - C$. A subset C of vertices of G is called a *cutset* if the removal of C from G disconnects G . We call C a *connecting set of G* if C is a cutset of G and the removal of C from G disconnects G into at least $|C| + 1$ connected components.

Arikati and Rangan gave an $O(n + m)$ -time algorithm for the path cover problem on interval graphs [1]. Manacher et. al. gave an $O(n \log n)$ -time algorithm for the Hamiltonian path cover problem on interval graphs [19]. The two papers were published in the the same volume of Information Processing Letters. They used the same greedy approach independently. We first review

their greedy approach. Assume that the input graph is given by an interval model I that is a set of n sorted intervals labeled by $1, 2, \dots, n$ in increasing order of their right endpoints. Notice that we do not distinguish an interval from its label. The left endpoint of interval x is denoted by $left(x)$ and the right endpoint by $right(x)$. Interval x is denoted by $(left(x), right(x))$. An interval x is said to *contain* another interval y if every point of y falls within the interior of $(left(x), right(x))$. For convenience, we need the following notation.

(1) For two distinct intervals x, y in I , x is said to be smaller than y , denoted by $x < y$, if $right(y)$ is to the right of $right(x)$, and y is said to be to the right of x , denoted by $x \ll y$, if $left(y)$ is to the right of $right(x)$.

(2) $s(I)$ denotes the interval in I with the leftmost right endpoint. The algorithm for the path cover problem on an interval model I uses a greedy principle to extend a path $Z_i = z_1 \rightarrow z_2 \rightarrow \dots \rightarrow z_k$, $k \geq 1$. Initially, $i = 1$ and Z_1 visits $s(I)$, i.e., $z_1 = s(I)$. Then, the unvisited neighbor of z_k with the leftmost right endpoint is chosen and Z_i is extended to visit it. If no such neighbor exists, then Z_i is stopped and a new path Z_{i+1} is started in the remaining graph from the smallest labeled unvisited interval if possible. To simplify the notation, we denote the algorithm and its output by Algorithm GP and $GP(I)$, respectively. For instance, given a set of sorted intervals shown in Fig. 1, Algorithm GP outputs two paths $Z_1 = i_1 \rightarrow i_2 \rightarrow i_3 \rightarrow i_4 \rightarrow i_6 \rightarrow i_5$ and $Z_2 = i_7 \rightarrow i_8 \rightarrow i_9 \rightarrow i_{10} \rightarrow i_{12} \rightarrow i_{11}$.

It is easy to see that Algorithm GP runs in $O(n + m)$ time. Chang et al. showed that Al-

gorithm GP can be done in $O(n)$ time and $O(n)$ space given a set of sorted intervals [4]. The paths in $GP(I)$ are called *greedy paths*. Arikati and Rangan proved by induction that $GP(I)$, the set of paths output by Algorithm GP is an optimal path cover of $G(I)$, the graph obtained from interval model I . Manacher et. al. proved that $G(I)$ has a Hamiltonian path if and only if $|GP(I)| = 1$ by showing that there exists a set C of intervals such that $G(I - C)$ has more than $|C| + 1$ connected components. Their proof implies an $O(n)$ -time algorithm for finding such a set C . Given a set of endpoint-sorted intervals, the number of connected components of the interval graph obtained from the set of input intervals can be computed in $O(n)$ time. Hence C can be considered as a certificate showing that $G(I)$ does not have any Hamiltonian path. Therefore the algorithm by Manacher et. al. for the Hamiltonian path problem on interval graphs can be easily extended to be a certifying algorithm. Unfortunately it is not easy to see if a certificate showing that $GP(I)$ is an optimal path cover of $G(I)$ exists from Arikati and Rangan's proof. Inspired by the proof of Manacher et. al., we give an $O(n)$ -time algorithm for computing a certificate showing that $GP(I)$ is indeed an optimal path cover of $G(I)$. Actually this certificate is a generalization of the certificate for showing that $G(I)$ does not have any Hamiltonian path. From $GP(I)$ we compute in $O(n)$ time a set C of intervals such that $G(I - C)$ has $|GP(I)| + |C|$ connected components. By pigeon hole principle, it is easy to see that such a set C is indeed a certificate showing that $GP(I)$ is an optimal path cover of $G(I)$.

Note that the path-start of a greedy path is the smallest labeled interval in the path. Define $L(Z)$ of a path Z to be the collection of intervals in Z which are larger than $end(Z)$, i.e., $L(Z) = \{x | x \in Z \text{ and } end(Z) < x\}$. A path Z is called a *monotone path* if and only if $L(Z) = \emptyset$. For instance, let $Z_1 = i_1 \rightarrow i_2 \rightarrow i_3 \rightarrow i_4 \rightarrow i_6 \rightarrow i_5$ be a path of $G(I)$ shown in Fig. 1, then $L(Z_1) = \{i_6\}$. The following lemma can be verified by the greedy rule of Algorithm GP and has been proved in [11].

Lemma 2.1. (Hung et al. [11]) *Suppose Z is a greedy path in $GP(I)$. Then,*

- (1) *an interval x is in Z if and only if $left(x)$ is to the left of $right(end(Z))$,*
- (2) *if intervals x and y do not intersect and $x, y \in Z$, then Z visits x before y if and only if $x \ll y$,*
- (3) *an interval x is in $L(Z)$ if and only if x contains $right(end(Z))$, and*
- (4) *an interval x is in $L(Z)$ if and only if x contains $end(Z)$.*

Given a greedy path Z , we can find a subset $C(Z)$ of Z so that the removal of $C(Z)$ from Z disconnects Z into $|C(Z)| + 1$ subpaths. The procedure was given in [11]. For readers' convenience, we describe it as follows:

Procedure GCS

Input: Z , a greedy path in $GP(I)$.

Output: $C(Z)$, the *greedy connecting set* of path Z , and $R(Z)$, the set of all of the sub-paths of Z obtained by removing $C(Z)$ from Z .

Method:

1. $S = Z$; $C(Z) = \emptyset$; $R(Z) = \emptyset$;
2. $z = end(Z)$; $CurrentPathEnd = end(Z)$;
3. **while** $z \neq start(S)$ **do**

4. $z = \text{predecessor}(z)$;
5. **if** $\text{CurrentPathEnd} < z$, **then**
6. let $S = S_1 \rightarrow z \rightarrow S_2$, where S_1 and S_2 are two subpaths of S ;
7. $C(Z) = C(Z) \cup \{z\}$; $R(Z) = R(Z) \cup \{S_2\}$;
8. $S = S_1$; $\text{CurrentPathEnd} = \text{end}(S_1)$;
 $z = \text{end}(S_1)$;
9. $R(Z) = R(Z) \cup \{S\}$;
10. **output** $C(Z)$ and $R(Z)$.

Since each interval is visited once, Procedure $\text{GCS}(Z)$ runs in $O(|Z|)$ time. For instance, given a greedy path $Z_1 = i_1 \rightarrow i_2 \rightarrow i_3 \rightarrow i_4 \rightarrow i_6 \rightarrow i_5$ of Fig. 1, Procedure GCS outputs $C(Z_1) = \{i_6\}$ and $R(Z_1) = \{i_1 \rightarrow i_2 \rightarrow i_3 \rightarrow i_4, i_5\}$. The following lemma shows some properties of $C(Z)$ and $R(Z)$ and is used in proving the correctness of our algorithm.

Lemma 2.2. (Hung et al. [11]) Suppose Z_1 is the first greedy path output by Algorithm GP given a set I of sorted intervals. Let $C(Z_1) = \{c_1, c_2, \dots, c_k\}$ and $R(Z_1) = \{Z_{1,1}, Z_{1,2}, \dots, Z_{1,k+1}\}$ be two sets output by Procedure $\text{GCS}(Z_1)$ such that $Z_1 = Z_{1,1} \rightarrow c_1 \rightarrow Z_{1,2} \rightarrow c_2 \rightarrow \dots \rightarrow c_k \rightarrow Z_{1,k+1}$. Then, the following statements hold:

- (1) Every path in $R(Z_1)$ is a monotone path.
- (2) $L(Z_1) \subseteq C(Z_1)$.
- (3) $Z_{1,1}, Z_{1,2}, \dots, Z_{1,k+1}$ are islands with respect to $C(Z_1)$ in $G(I)$.
- (4) Intervals visited by $Z_{1,j}$ are to the right of intervals visited by $Z_{1,i}$ if and only if $1 \leq i < j \leq k+1$.
- (5) If $I \setminus Z_1 \neq \emptyset$, then $I \setminus Z_1$ is an island with respect

to $L(Z_1)$ in $G(I)$ and $x \ll y$ for $x \in Z_1 \setminus L(Z_1)$ and $y \in I \setminus Z_1$.

- (6) c_i contains every interval of $Z_{1,i+1}$ for $1 \leq i \leq k$.

The following lemma was the basis in proving the correctness of Algorithm GP and was given in [11].

Lemma 2.3. [Hung et al. [11]] Let $C(GP(I)) = \cup_{Z_i \in GP(I)} C(Z_i)$ and $R(GP(I)) = \cup_{Z_i \in GP(I)} R(Z_i)$, where Z_i is the i -th greedy path output by Algorithm GP given a set I of sorted intervals, $C(Z_i)$ is the greedy connecting set of Z_i , and $R(Z_i) = Z_i \setminus C(Z_i)$. Then, the following statements hold:

- (1) If $|GP(I)| \geq 2$, then the removal of $C(Z_1)$ from $G(I)$ results in at least $|C(Z_1)| + 2$ connected components.
- (2) Each element of $R(GP(I))$ is a connected component of $G(I \setminus C(GP(I)))$.
- (3) $|R(GP(I))| = |C(GP(I))| + |GP(I)|$.

Let I be the set of intervals shown in Fig. 1. Then, $C(GP(I)) = \{i_6, i_{12}\}$ and $R(GP(I)) = \{i_1 \rightarrow i_2 \rightarrow i_3 \rightarrow i_4, i_5, i_7 \rightarrow i_8 \rightarrow i_9 \rightarrow i_{10}, i_{11}\}$. By Lemma 2.3, we can find a subset $C(GP(I))$ of I such that $|R(GP(I))| = |C(GP(I))| + |GP(I)|$. Each path in $R(GP(I))$ is a connected component of $G(I \setminus C(GP(I)))$. From above discussion, $C(GP(I))$ can be considered as a certificate showing that $GP(I)$ is an optimal path cover of the interval graph obtained from I . It is easy to see that $C(GP(I))$ can be computed in $O(n)$ time. Thus we have a certifying algorithm for the path cover problem on interval graphs.

References

- [1] S.R. Arikati and C. Pandu Rangan, Linear algorithm for optimal path cover problem on interval graphs, *Inform. Process. Lett.* **35** (1990) 149–153.
- [2] A.A. Bertossi and M.A. Bonuccelli, Hamiltonian circuits in interval graph generalizations, *Inform. Process. Lett.* **23** (1986) 195–200.
- [3] F.T. Boesch and J.F. Gimpel, Covering the points a digraph with point-disjoint paths and its application to code optimization, *J. ACM* **24** (1977) 192–198.
- [4] M.S. Chang, S.L. Peng and J.L. Liaw, Deferred-query: an efficient approach for some problems on interval graphs, *Networks* **34** (1999) 1–10.
- [5] G.J. Chang, Corrigendum to “The path-partition problem in block graphs” [Information Processing Letters 52 (1994) 317–322], *Inform. Process. Lett.* **83** (2002) 293.
- [6] P. Damaschke, The Hamiltonian circuit problem for circle graphs is NP-complete, *Inform. Process. Lett.* **32** (1989) 1–2.
- [7] P. Damaschke, J.S. Deogun, D. Kratsch and G. Steiner, Finding Hamiltonian paths in comparability graphs using the bump number algorithm, *Order* **8** (1992) 383–391.
- [8] M.R. Garey and D.S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, San Francisco, CA, 1979.
- [9] M.C. Golumbic, *Algorithmic Graph Theory and Perfect Graphs*, Academic Press, New York, 1980.
- [10] P. Hell and J. Huang, Certifying LexBFS recognition algorithms for proper interval graphs and proper interval bigraphs, *SIAM J. Discrete Math.* **18** (2004) 554–570.
- [11] Ruo-Wei Hung and Maw-Shang Chang, Solving the path cover problem on circular-arc graphs by using an approximation algorithm, *Discrete Appl. Math.* **154** (2006) 76–105.
- [12] Ruo-Wei Hung and Maw-Shang Chang, Finding a minimum path cover of a distance-hereditary graph in polynomial time *Discrete Appl. Math.* **155** (2007) 2242–2256.
- [13] A. Itai, C.H. Papadimitriou and J.L. Szwarcfiter, Hamiltonian paths in grid graphs, *SIAM J. Comput.* **11** (1982) 676–686.
- [14] Haim Kaplan and Yahav Nussbaum, Certifying algorithms for recognizing proper circular-arc graphs and unit circular-arc graphs, in *Graph-theoretic Concepts in Computer Science*, Lecture Notes in Comput. Sci. 4271, Springer, Berlin, 2006, pp. 11–16.
- [15] M.S. Krishnamoorthy, An NP-hard problem in bipartite graphs, *SIGACT News* **7** (1976) 26.
- [16] D. Kratsch, R. McConnell, K. Mehlhorn, and J.P. Spinrad, Certifying algorithms for recognizing interval graphs and permutation graphs. In *SODA*, 2003, pp. 158–167.

- [17] Dieter Kratsch, Ross M. McConnell, Kurt Mehlhorn, and Jeremy P. Spinrad, Certifying algorithms for recognizing interval graphs and permutation graphs, *Siam J. Comput.* **36** (2006) 326–353.
- [18] R. Lin, S. Olariu, and G. Pruesse, An optimal path cover algorithm for cographs, *Comput. Math. Appl.* 30 (1995) 75–83.
- [19] G.K. Manacher, T.A. Mankus and C.J. Smith, An optimum $\Theta(n \log n)$ algorithm for finding a canonical Hamiltonian path and a canonical Hamiltonian circuit in a set of intervals, *Inform. Process. Lett.* **35** (1990) 205–211.
- [20] Kurt Mehlhorn, Arno Eigenwillig, Kanela Kanegossi, Dieter Kratsch, Ross McConnell, Uli Meyer, and Jeremy Spinrad, Certifying Algorithms (A Paper under Construction), www.mpi-inf.mpg.de/mehlhorn/ftp/CertifyingAlgorithms.pdf.
- [21] S. Moran and Y. Wolfstahl, Optimal covering of cacti by vertex disjoint paths, *Theoret. Comput. Sci.* 84 (1991) 179–197.
- [22] S. Pinter and Y. Wolfstahl, On mapping processes to processors, *Internat. J. Parallel Programming* 16 (1987) 1–15.
- [23] R. Srikant, R. Sundaram, K.S. Singh, and C.P. Rangan, Optimal path cover problem on block graphs and bipartite permutation graphs, *Theoret. Comput. Sci.* 115 (1993) 351–357.
- [24] P.K. Wong, Optimal path cover problem on block graphs, *Theoret. Comput. Sci.* 225 (1999) 163–169.
- [25] J.H. Yan and G.J. Chang, The path-partition problem in block graphs, *Inform. Process. Lett.* 52 (1994) 317–322.
- [26] H.G. Yeh and G.J. Chang, The path-partition problem in bipartite distance-hereditary graphs, *Taiwanese J. Math.* 2 (1998) 353–360.

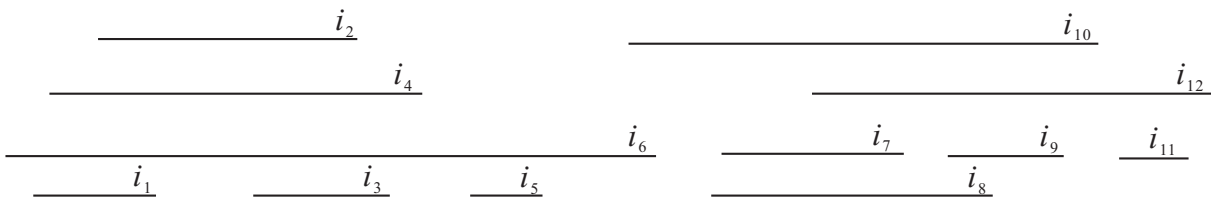


Figure 1: An interval model I of 12 sorted intervals.